

De PDFs escaneados a informes legales con IA

OCR open-source, GPU y agentes de IA: un caso real de *legaltech*

Abigail Quispe Alexander Quispe

abigail.quispe@pucp.edu.pe · aquisper@caltech.edu

Universidad del Pacífico

1 de junio de 2026

¿Qué vamos a ver hoy?

- **Quiénes somos** como abogados, qué problema enfrentamos y nuestro **objetivo**.
- Un **problema real**: analizar bases de licitaciones que vienen como PDF escaneado.
- Cómo elegir entre **API de pago** y **open-source**.
- Por qué **conocer tu hardware** (CPU, RAM, GPU) decide qué puedes correr.
- Un **fracaso instructivo**: un modelo de IA que no entró en la GPU.
- La solución: **PaddleOCR + GPU + un agente de IA** para el análisis.
- Cómo lo convertimos en una **app** y las **lecciones transferibles**.

Objetivo: no es memorizar comandos, sino entender cómo se toman las decisiones técnicas.

¿Quiénes somos y qué hacemos?

- Somos abogados del área de **licitaciones** de una empresa de telecomunicaciones.
- Cuando el Estado necesita contratar un servicio (p.ej. *internet dedicado*), convoca un **concurso público** y publica las **bases**.
- Antes de ofertar, **revisamos esas bases a fondo** para defender la posición de la empresa y asegurar reglas justas.

¿Qué son las Consultas y Observaciones?

Toda licitación tiene una **etapa formal** para que los participantes presenten:

- **Consultas** — pedidos de *aclaración* ante dudas o ambigüedades de las bases.
- **Observaciones** — *cuestionamientos* cuando una regla vulnera la ley o las Bases Estándar.

Clave

Tienen **plazo perentorio** y la Entidad debe responderlas e integrar las bases. Es la oportunidad de corregir las reglas *antes* de competir.

¿Qué está en juego?

- Unas bases mal diseñadas pueden “**direccionar**” la contratación hacia un proveedor (p. ej. exigir una **marca** específica).
- Si no observamos a tiempo: competimos en una **cancha inclinada**. . . o quedamos fuera.
- Una buena observación **abre la competencia**, nivela el terreno y a veces **decide si podemos ganar**.

Es trabajo jurídico de alto valor — y muy intensivo en lectura.

Nuestro objetivo inicial

El encargo

Reducir el **costo** y el **tiempo** de producir el informe de Consultas y Observaciones, sin perder calidad.

En concreto:

- 1 **Leer** las bases escaneadas de forma barata o gratuita (OCR).
- 2 **Apoyar el análisis** jurídico con IA (borrador de consultas/observaciones).
- 3 **Generar el informe** con el formato oficial, listo para presentar.

Meta: subir un PDF y obtener un borrador en **minutos**.

El recorrido: los pasos que seguimos

- 1 Buscar una alternativa **open-source** al OCR de pago.
- 2 Evaluar el **hardware** disponible (¿qué podemos correr?).
- 3 Probar un **LLM de visión** local... (falló por memoria).
- 4 **Pivotar** a un OCR dedicado: **PaddleOCR**.
- 5 **Acelerar** con la **GPU** (40×).
- 6 Empaquetar todo en una **app** (OCR + IA + PDF).

Spoiler: el camino incluyó un fracaso, y de ahí salió la mejor lección.

El problema de partida

- Somos un estudio que formula **Consultas y Observaciones** a bases de concursos públicos.
- Las bases (sobre todo el TDR) llegan como **imágenes escaneadas** dentro del PDF: *no se puede copiar el texto*.
- Estábamos usando la **API de Anthropic** para leerlas
→ **caro** al hacerlo a gran escala.

La pregunta

¿Existe una alternativa **open-source y gratuita** para hacer este OCR en mi propia computadora?

Concepto clave: dos tipos de PDF

PDF con capa de texto

- El texto es **seleccionable**.
- Se extrae **gratis e instantáneo** (p. ej. con pdfjs).

PDF escaneado (imagen)

- Es una **foto**: no hay texto.
- Requiere **OCR** (reconocimiento óptico de caracteres).

*Regla de oro: usa OCR **solo** en las páginas que son imagen; el resto, extracción directa.*

El menú de opciones open-source

Herramienta	Tipo	Nota
Tesseract / Tesseract.js	OCR clásico	Liviano; sufre con tablas/escaneos malos
PaddleOCR	OCR moderno (CNN)	Rápido, multilenguaje, buen español
Surya / docTR	OCR <i>deep learning</i>	Muy preciso
Marker / MinerU	PDF → Markdown	Conserva estructura/tablas
Qwen2.5-VL, MiniCPM-V	<i>LLM de visión</i>	Calidad tipo Claude, pero pesados

Criterio

No existe “la mejor”. Depende de: calidad de los documentos, idioma, y **el hardware que tienes**.

Antes de elegir: conoce tu hardware

La máquina del caso:

- CPU Intel i7 (6 núcleos)
- 32 GB de RAM
- GPU **NVIDIA GTX 1660 Ti — 6 GB VRAM**

El dato que lo decide todo

La **VRAM** (memoria de la GPU) determina qué modelos de IA puedes cargar.

6 GB es modesto: alcanza para modelos pequeños, no para los grandes.

Intento 1: un LLM de visión local (Ollama + Qwen2.5-VL)

- Idea: correr un modelo “tipo Claude” en local con **Ollama**, gratis.
- Instalamos todo, descargamos el modelo (3B)...y al correrlo: **lentísimo**.
- Diagnóstico **leyendo los logs** del servidor:

Lo que decía el log

```
offloaded 0/37 layers to GPU  
compute graph ... size="6.7 GiB"  
total memory ... size="10.1 GiB"
```

El modelo necesitaba ~**10 GiB** pero la GPU solo tiene **6 GB** → cayó a CPU → **una página densa tardaba +10 min.**

Lección: “modelo de visión” $\neq$ “OCR dedicado”

LLM de visión (Qwen 3B)

- Razona sobre la imagen.
- **Enorme** en memoria (~ 10 GiB con la imagen).
- **X** No entra en 6 GB.

OCR dedicado (PaddleOCR)

- Solo reconoce caracteres.
- Modelos **diminutos** (cientos de MB).
- **✓** Entra de sobra en 6 GB.

Moraleja: usa la herramienta del tamaño del problema. No traigas un cañón para matar una mosca.

- Cambiamos al OCR dedicado. Resultado: transcripción **fiel, con tildes**, en CPU y **gratis**.
- Detalles que aprendimos en el camino:
 - Un *bug* conocido en CPU se resuelve con un parámetro (`enable_mkldnn=False`).
 - Las **tablas apaisadas** (giradas 90°) salían como basura → se activa la **corrección de orientación**.

Antes vs. después (página girada)

Antes: SOSORA00S CS 00RSOD... (ilegible)

Después: N° CONTRATO | OBJETO | CLIENTE | MONTO... (tabla legible)

CPU vs. GPU: el momento “wow”

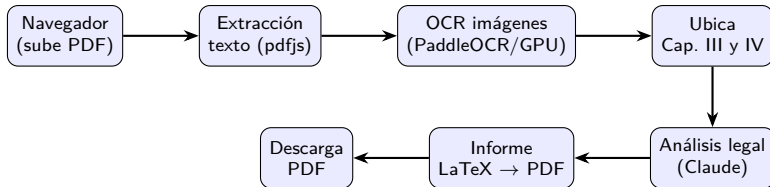
Instalamos la versión GPU de PaddleOCR (paddlepaddle-gpu). Mismo modelo, misma página:

	CPU	GPU (1660 Ti)
Una página densa	~45 s	~ 1 s
Rango de ~42 páginas	~32 min	~ 1 min

¿Por qué ahora **SÍ** ayudó la GPU?

Porque los modelos de PaddleOCR **sí entran** en 6 GB. La GPU acelera lo que cabe; no hace milagros con lo que no cabe (como el LLM de visión).

Arquitectura de la app (todo local)



Servidor local (Node/Express) que orquesta: Python (OCR), un agente de IA (análisis) y LaTeX (PDF).

Decisión de diseño clave: ¿qué hace cada parte?

OCR **local** (PaddleOCR)

- Tarea **repetitiva y de volumen**.
- Gratis, en tu GPU. ✓

Análisis **con IA** (Claude)

- Tarea de **razonamiento jurídico** de alto valor.
- Pocos tokens, gran impacto. ✓

Principio: pon el trabajo barato y masivo en local; reserva el modelo caro para lo que de verdad agrega valor.

Truco: usar Claude sin clave de API

- El **análisis** se puede hacer con la **suscripción** de Claude Code, sin pagar API aparte.
- Se invoca en modo “headless” (sin interfaz) desde el servidor:

Llamada en modo headless

```
claude -p "Analiza el TDR y devuelve JSON" --output-format json
```

- Devuelve la respuesta lista para que el programa la procese (JSON estructurado).
- *(Trade-offs: depende del cupo de la suscripción y de tener Claude Code instalado.)*

Optimizar: hacer OCR solo de lo necesario

- Solo analizamos los **Capítulos III y IV** (~42 págs), no las 110.
- Estrategia: **OCR progresivo** que **se detiene** al detectar el inicio del Cap. V.
- **Bug encontrado:** Python “guarda en buffer” su salida cuando otro programa la captura → nuestro detector de “ya llegamos” no se enteraba.
- **Solución:** leer el **archivo** que el OCR va escribiendo, en vez de su consola.

Lección: los detalles “tontos” (buffering, formatos) son la mitad del trabajo real de programar.

Responsabilidad profesional: humano en el centro

- La IA genera un **borrador**, no la versión final.
- La app verifica que cada **cita** exista **literal** en el texto (marca lo “sin verificar”).
- El abogado **revisa y edita** antes de generar el PDF.

Por qué importa

En documentos legales (y en general), la IA es un **copiloto**, no un piloto automático. La responsabilidad sigue siendo humana.

Buenas prácticas de ingeniería que aplicamos

- **No subir al repositorio** lo que no debe: dependencias gigantes (node_modules, .venv de 3.9 GB) ni **secretos** (.env). → .gitignore.
- GitHub **rechaza archivos >100 MB**: las dependencias se **reinstalan**, no se versionan.
- Guardado **incremental** y **reanudable**: si un proceso largo se corta, no se pierde el avance.
- **Diagnosticar por logs** antes de adivinar.

Nuestro output: un ejemplo real

El sistema produce items como este (Capítulo III, p. 32):

Observación

Cita de las bases: *«basado en tecnología ASIC... Complete Content Protection»*

Motivación: es **terminología propietaria** de un fabricante; **direcciona** la contratación hacia una marca. Solicitamos describir las funcionalidades de modo **neutral**, admitiendo soluciones equivalentes.

Norma vulnerada: Art. 44.6 del Reglamento; Art. 5.1 h) y j) de la Ley N.º 32069.

Todo en una **tabla editable** que el abogado revisa antes de generar el PDF.

De principio a fin

PDF escaneado de 110 páginas → informe de **Consultas y Observaciones** en PDF, en **~4 minutos**, **sin costo de API** y en la propia computadora.

- OCR local gratis (PaddleOCR), **acelerado 40×** con la GPU.
- Análisis jurídico con un agente de IA usando la suscripción.
- Una app que un colega puede usar subiendo un PDF.

Lecciones transferibles (lo que de verdad importa)

- 1 **Define el problema** antes de elegir la herramienta.
- 2 **Conoce tus límites** (hardware/VRAM): deciden qué es posible.
- 3 **La herramienta del tamaño del problema** (OCR dedicado vs LLM gigante).
- 4 **Híbrido**: lo barato/masivo en local; el modelo caro solo donde agrega valor.
- 5 **Itera y diagnostica**: los fracasos enseñan (Qwen → PaddleOCR).
- 6 **Humano en el centro**: la IA propone, la persona decide.

¡Gracias!

Construyamos algo en clase.